HBA TECH ET TRADE · DOSSIER TECHNIQUE

Architecture du backend

HbaExpress Marketplace — monolithe modulaire .NET 8, place de marché multi-vendeurs pour le Bénin / UEMOA.

Document reconstitué par lecture directe du dépôt `marketPlace` : solution, registres de modules, contrats, handlers d'événements, composition Docker et chaîne de livraison.

25	5	~95 k
MODULES COMPOSÉS	HÔTES EXÉCUTABLES	LIGNES C#

614	105	287
ROUTES HTTP	MIGRATIONS EF	TESTS

Périmètre : dépôt `marketPlace` uniquement (backend). État décrit : l'arbre de travail au 8 septembre 2026, branche `feat/figer-bareme-sur-ligne-commande` (non fusionnée) sur `e15c802`.

Sommaire

1	**Vue d'ensemble** — style architectural, décisions structurantes, chiffres clés
2	**Topologie applicative** — les cinq hôtes et le partage du même noyau
3	**Anatomie d'un module** — les quatre couches et la règle de dépendance
4	**Le noyau partagé** — BuildingBlocks : Domain, Application, Infrastructure, IntegrationEvents
5	**Communication inter-modules** — le canal synchrone et le canal asynchrone
6	**L'outbox transactionnelle** — garanties, backoff, lettres mortes, contrainte d'instance unique
7	**Cartographie des modules** — schéma, agrégats, volumétrie, événements
8	**Le saga de commande** — du panier à la livraison, compensations comprises
9	**Le flux financier** — commission, séquestre, portefeuilles, règlement
10	**Persistance** — un schéma par module, migrations, concurrence optimiste
11	**Sécurité** — chaîne d'autorisation en quatre couches
12	**Intégrations externes** — paiements, notifications, médias, référentiel géographique
13	**Observabilité** — logs, traces, métriques, sondes
14	**Déploiement** — topologie VPS, Traefik, chaîne CI/CD
15	**Garde-fous** — tests d'architecture et stratégie de test
16	**Limites connues et trajectoire** — ce qui bloque la montée en charge
A	**Annexe** — inventaire des événements d'intégration et de leurs abonnés

Méthode

Ce document décrit l'arbre de travail au 8 septembre 2026. Il inclut donc les corrections de la branche `feat/figer-bareme-sur-ligne-commande`, non encore fusionnée : le figeage du barème sur la ligne de commande (§ 9) et le nettoyage des dossiers de modules vides (§ 7).

Chaque affirmation provient de la lecture du code, pas de la documentation existante du dépôt : registre des modules (`ModuleRegistry`), enregistrements de handlers dans les `*ModuleInstaller`, contrats publics des modules, composition `docker-compose.prod.yml`, workflow GitHub Actions et tests d'architecture. Les chiffres sont mesurés sur l'arborescence `src/` hors `bin/` et `obj/`.

1 · Vue d'ensemble

> Un monolithe modulaire .NET 8 : un seul dépôt, un seul graphe de compilation, une seule base PostgreSQL — mais vingt-cinq modules qui ne se parlent que par contrats, et dont chacun pourrait être extrait sans réécriture.

1.1 · Le style et ce qu'il achète

Le choix n'est ni « monolithe » ni « microservices », mais la position intermédiaire qui garde les propriétés utiles des deux. Du monolithe : une transaction unique par commande, un déploiement atomique, aucun coût réseau entre modules, un débogage local complet. Des microservices : des frontières explicites, testées mécaniquement, et une communication qui passe déjà par des événements sérialisés — de sorte que le jour où un module doit sortir, le contrat existe déjà.

La frontière est matérialisée à trois niveaux simultanément, ce qui est la seule manière qu'elle ne s'érode pas :

- **Au niveau du code** — un module n'expose qu'un projet `*.Contracts`. Ses types internes (agrégats, DbContext, handlers) sont `internal` ou hors référence.
- **Au niveau de la base** — un schéma PostgreSQL par module, sans clé étrangère ni jointure inter-schémas. L'extraction devient une opération mécanique.
- **Au niveau des tests** — quatorze suites `NetArchTest` échouent la compilation de la CI si une couche franchit une frontière.

1.2 · Les décisions structurantes

Décision	Ce qu'elle implique
Monolithe modulaire 25 modules, 1 processus par hôte	Les modules sont composés *in-process*. Aucun appel réseau entre eux, donc aucune latence ni panne partielle à gérer — au prix d'un déploiement solidaire.
CQRS léger sur MediatR MediatR 12.4.1 (dernière ligne MIT)	Commandes et requêtes explicites, un handler par cas d'usage, un pipeline transverse commun (journalisation puis validation).
Pattern Result jamais d'exception pour un échec attendu	`Result` / `Result<T>` traversent toutes les couches. Une erreur métier est une valeur, transformée en `ProblemDetails` au bord HTTP.
DDD tactique AggregateRoot, Entity, ValueObject, Money	L'agrégat est le point d'entrée transactionnel et le seul à accumuler des *domain events*, collectés au `SaveChanges`.
Outbox transactionnelle une table par schéma de module	L'événement d'intégration est écrit dans la même transaction que le changement d'état : garantie « au moins une fois », sans bus externe.
Backend for Frontend 4 BFF + 1 API	Chaque client a son hôte, ses routes et sa surface d'exposition. L'API monolithe n'est plus qu'un migrateur et un récepteur de webhooks.
Schéma par module 25 schémas, 1 base	Isolation logique sans coût opérationnel. Une base à sauvegarder, vingt-cinq périmètres à respecter.
XOF par défaut Money.DefaultCurrency	Le marché visé est l'UEMOA. La devise est portée par un value object partagé, arrondi à deux décimales, addition refusée entre devises différentes.

1.3 · Chiffres du dépôt

Code	Valeur
Fichiers C# (`src/`)	1 197
Lignes C# (`src/`)	95 481
Projets .csproj	109
Modules composés	25
Schémas PostgreSQL	25
Migrations EF Core	105

Surface & tests	Valeur
Routes HTTP (5 hôtes)	614
Agrégats du domaine	38
Événements d'intégration	38
Abonnements à ces événements	47
Projets de test	29
Cas de test	287

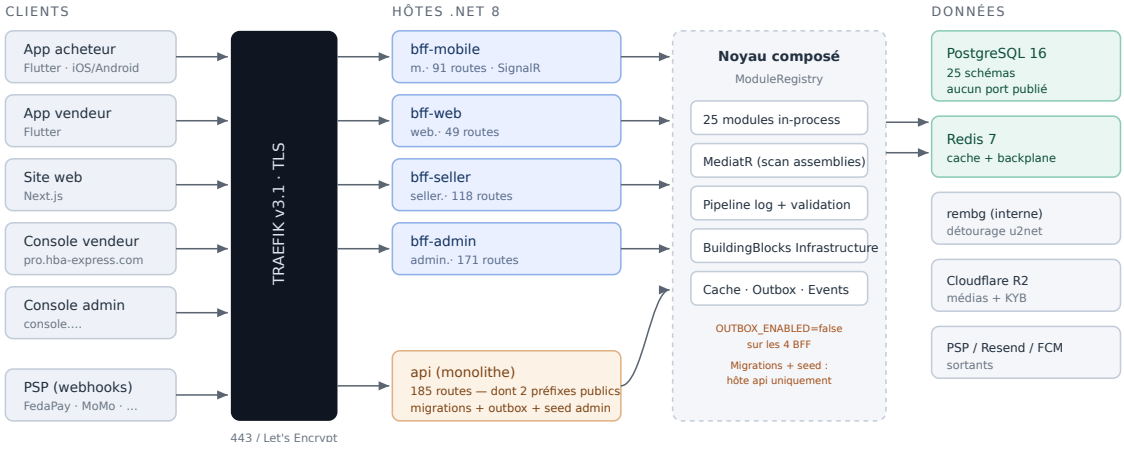
1.4 · Socle technique

Domaine	Choix
Plateforme	.NET 8 (`net8.0`), C# `latest` , `Nullable` activé, `TreatWarningsAsErrors` , `InvariantGlobalization`
Versions	Central Package Management — une seule source de vérité (`Directory.Packages.props`)
Médiation	MediatR 12.4.1 · FluentValidation 11.9.2
Persistance	EF Core 8.0.8 · Npgsql 8.0.8 · PostgreSQL 16
Cache & temps réel	Redis 7 (cache distribué + backplane SignalR)
Sécurité	JWT Bearer · BCrypt.Net-Next 4.0.3 · Otp.NET (TOTP)
Images	SkiaSharp 2.88.8 (MIT, sans dépendance système) + service `rembg` local
Notifications	FirebaseAdmin 3.1.0 (FCM HTTP v1) · Resend (e-mail transactionnel)
Observabilité	Serilog 8 (JSON compact) · OpenTelemetry 1.15.x (OTLP) · HealthChecks Npgsql/Redis
Tests	xUnit 2.9 · FluentAssertions 6.12 · Testcontainers.PostgreSql 3.9 · NetArchTest 1.3.2

> **UNE NOTE SUR LES LICENCES**
>
> Deux choix de dépendances sont contraints par la licence et non par la technique : MediatR est figé en 12.x, dernière ligne MIT ; SkiaSharp est préféré à ImageSharp, passé en licence commerciale au-delà de la 2.0 — payante pour une place de marché. Sur un produit destiné à être exploité commercialement, ce n'est pas un détail.

2 · Topologie applicative

> Cinq exécutables, un seul noyau. Chaque hôte compose exactement les mêmes vingt-cinq modules en mémoire ; ce qui les distingue, c'est la surface HTTP qu'ils exposent et ce qu'ils ont le droit de faire tourner en tâche de fond.



Cinq hôtes derrière un seul point d'entrée TLS, un noyau applicatif identique, une base et un cache mutualisés. Les cinq hôtes partagent la **même clé de signature JWT** — d'où la seconde couche d'autorisation par rôle du § 11.

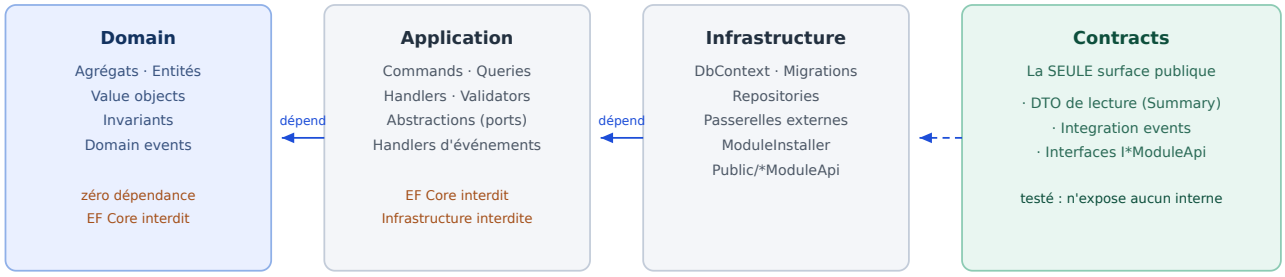
> **POURQUOI L'API N'EST PRESQUE PLUS EXPOSÉE**
>
> Aucun client n'appelle l'API monolithe : l'app acheteur parle à `m.`, l'app vendeur à `seller.`, les consoles à `admin.` et `seller.`. La seule chose qui doive la joindre depuis l'extérieur, ce sont les rappels des prestataires de paiement. La règle Traefik l'exprime littéralement, et cela constitue une défense en profondeur : un `RequireAuthorization()` oublié demain sur une route de l'API ne serait de toute façon pas atteignable depuis Internet. La contrepartie est explicite dans le compose : **tout nouvel endpoint devant être appelé de l'extérieur doit être ajouté à cette règle**, faute de quoi il répondra 404 — et le symptôme (« le PSP ne nous rappelle jamais ») ne pointera pas vers ce fichier.

2.1 · Rôle de chaque hôte

Hôte	Routes	Responsabilité
`Marketplace.Api` **particulier**	185	Monolithe historique. En production sa surface publique est **réduite à deux préfixes** — `/api/payments/webhooks` et `/api/payments/return` — par la règle de routage Traefik. Il reste le seul hôte à appliquer les migrations, semer le premier administrateur et drainer les outbox.
`Bff.Mobile`	91	Application acheteur : catalogue, découverte, panier, checkout orchestré en un appel, commandes, messagerie temps réel (SignalR, jeton passé sur le handshake), liens profonds, contenu éditorial.
`Bff.Web`	49	Vitrine web. Surface volontairement la plus étroite : catalogue, panier, checkout, compte, commandes.
`Bff.Seller`	118	Espace vendeur : inscription et KYB, boutique, offres, stock, expédition, retours, litiges, avis, finances et portefeuille, notifications.
`Bff.Admin`	171	Back-office : modération, utilisateurs et rôles, catalogue de référence, finance, vendeurs et KYB, expédition, supervision — dont la lecture et le rejeu des **lettres mortes d'outbox** des 25 modules.

3 · Anatomie d'un module

> Quatre projets, une seule direction de dépendance. La règle tient en une phrase : rien de ce qui contient de la règle métier ne connaît la base de données.



Un module

A qui a besoin de **B** ne référence QUE `B.Contracts` — jamais son Domain, son Application ni son Infrastructure. Cette règle est vérifiée par les suites `NetArchTest` : elle casse la CI, pas seulement la revue de code.

Les quatre projets d'un module et le sens unique des dépendances.

3.1 · Le pipeline d'une commande

Chaque cas d'usage traverse la même chaîne, identique dans les cinq hôtes :

```
// Ordre voulu : la journalisation englobe la validation.
services.AddTransient(typeof(IPipelineBehavior<,>), typeof(LoggingBehavior<,>));
services.AddTransient(typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));
```

Puis le handler, puis `SaveChangesAsync` — qui n'est pas un simple *flush* : c'est l'unité de travail qui, dans l'ordre, dispatche les *domain events* des agrégats modifiés, draine la file d'événements d'intégration vers la table outbox du module, et écrit le tout en une seule transaction.

3.2 · Contrats CQRS

Type	Signature
`ICommand`	`IRequest<Result>` — effet de bord seul
`ICommand<T>`	`IRequest<Result<T>>` — renvoie par exemple l'identifiant créé
`IQuery<T>`	`IRequest<Result<T>>` — lecture pure

Aucune commande ne renvoie une exception pour un échec attendu. `Result.Failure(Error.Conflict(...))` remonte jusqu'au bord HTTP, où `ApiResults` le traduit en `ProblemDetails` normalisé.

4 · Le noyau partagé — `BuildingBlocks`

> Quatre projets, trente-huit fichiers. Tout ce qui serait recopié vingt-cinq fois, et rien d'autre : le noyau ne contient aucune règle métier.

Projet	Contenu
`.Domain`	**Primitives** — `Entity<TId>`, `AggregateRoot<TId>` (seul porteur de domain events), `ValueObject`, `Money` (XOF par défaut, arrondi 2 décimales, addition refusée entre devises). **Results** — `Result`, `Result<T>`, `Error` avec conversions implicites. **Geography** — le référentiel du Bénin : 12 départements, 77 communes, codes stables sans accent (`abomey-calavi`), libellés accentués séparés. Il vit ici et non dans un BFF parce que deux BFF le servent, et qu'une charge utile recopiée diverge au premier ajustement.
`.Application`	**Messaging** — les contrats CQRS. **Behaviors** — journalisation et validation. **Abstractions** — `ICacheService`, `IUnitOfWork`, `IDomainEventHandler`. **Observability** — abstractions de métriques et unités monétaires. **Pagination** — `PagedResult`.
`.Infrastructure`	**Persistence** — `ModuleDbContext`, la classe mère de tous les DbContext. **Outbox** — huit fichiers : message, configuration EF, file scopée, processeur, politique de réessai, enregistrement conditionnel. **Events** — dispatch des domain events et des integration events. **Caching** — `DistributedCacheService` (et une variante neutre). **Modularity** — `IModuleInstaller`. **Files** — `FileSignature`, contrôle du type réel des téléversements.
`.IntegrationEvents`	Le type de base `IntegrationEvent` et l'interface `IIntegrationEventHandler<T>`. Projet volontairement minuscule : c'est la dépendance commune que tous les `Contracts` peuvent référencer sans se contaminer.

4.1 · `ModuleDbContext` — l'unité de travail

Chaque module dérive de cette classe, qui lui apporte l'isolation par schéma, la table outbox locale et la séquence de sauvegarde :

```
public override async Task<int> SaveChangesAsync(CancellationToken ct = default)
{
    // 1. Dispatch des domain events : les handlers peuvent mettre
    //    des integration events en file.
    await DispatchDomainEventsAsync(ct);

    // 2. Draine la file vers l'outbox LOCAL (ce DbContext, ce schéma),
    //    de sorte que l'event et le changement d'état soient persistés ensemble.
    DrainIntegrationEventsToOutbox();

    return await base.SaveChangesAsync(ct);
}
```

> **LA RÈGLE D'OR DE LA PERSISTANCE**
>
> « Un module ne lit et n'écrit que dans son propre schéma. Pas de `JOIN` ni de clé étrangère inter-schémas. » C'est ce qui rend l'extraction d'un module mécanique plutôt que chirurgicale : sortir `Settlement` vers son propre service reviendrait à déplacer un schéma et à remplacer le dispatch in-process par un consumer — le code métier ne bougerait pas.

4.2 · `IModuleInstaller` — la composition

Chaque module fournit un installateur qui déclare son nom, son assembly Application (scannée par MediatR) et enregistre son DbContext, ses repositories, ses handlers d'événements, son API publique et son outbox. `ModuleRegistry` est **le seul endroit du dépôt qui connaît l'ensemble des modules** — et il ne dépend que de leurs projets Infrastructure d'enregistrement, jamais de leurs internes. Ajouter un module, c'est ajouter une ligne à ce tableau et une référence au `.csproj` de l'hôte.

5 · Communication inter-modules

> Deux canaux, deux usages, aucun troisième. Le synchrone quand il faut une réponse maintenant pour décider ; l'asynchrone quand une chose est arrivée et que d'autres modules doivent en tirer les conséquences.

Canal synchrone — `I*ModuleApi`

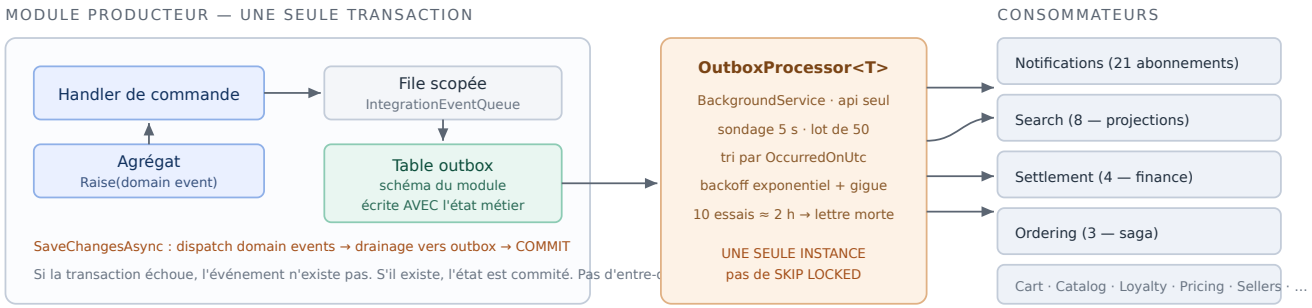
Seize interfaces déclarées dans les `Contracts`, implémentées dans le `Infrastructure/Public/` du module propriétaire. Lecture seule ou opération bornée, appelée dans la transaction de l'appelant.

`ICartModuleApi` · `ICatalogModuleApi` · `ICommissionModuleApi` · `IFraudModuleApi` · `IIdentityModuleApi` · `IInventoryModuleApi` · `IMessagingModuleApi` · `IOfferModuleApi` · `IOrderingModuleApi` · `IPaymentsModuleApi` · `IPayoutModuleApi` · `IPricingModuleApi` · `IReviewsModuleApi` · `ISellerModuleApi` · `IShippingModuleApi` · `ITaxModuleApi`

Canal asynchrone — *integration events*

Trente-huit événements publiés, quarante-sept abonnements. Le producteur ignore ses consommateurs : il met l'événement en file, le `SaveChanges` le persiste dans son outbox, le processeur le publie plus tard.

Le dispatch est aujourd'hui **in-process**. Le contrat étant un événement sérialisé en JSON avec un nom de type stable (`FullName, AssemblyName` sans version), le remplacement par un consumer Kafka ne toucherait aucun handler métier.



Garantie : « au moins une fois ». Les handlers doivent donc être idempotents — et ils le sont explicitement là où l'argent est en jeu : `AccrueEarningsOnOrderConfirmedHandler` commence par `ExistsForOrderAsync` et sort si le gain a déjà été comptabilisé.

Le chemin d'un événement d'intégration, de l'agrégat qui le lève au module qui le consomme.

6 · L'outbox transactionnelle

> C'est le mécanisme le plus critique du système : il porte la cohérence entre modules. Il est aussi celui qui impose la contrainte de déploiement la plus forte.

6.1 · Structure d'un message

Champ	Rôle
`Type`	Nom de type stable `FullName, AssemblyName` — sans version, pour survivre aux mises à jour.
`Content`	Charge utile JSON de l'événement.
`OccurredOnUtc`	Horodatage métier ; clé de tri du lot.
`ProcessedOnUtc`	`null` tant que non publié.
`AttemptCount`	Nombre d'échecs de publication.
`NextAttemptAtUtc`	Temporisation. C'est ce champ qui **sort un message en échec du lot courant**.
`DeadLetteredOnUtc`	Mise en lettre morte : le message ne sera plus jamais rejoué automatiquement.
`Error`	Message de la dernière erreur, tronqué à 2 000 caractères.

6.2 · Politique de réessai

Paramètre	Valeur
Intervalle de sondage	5 secondes
Taille de lot	50 messages, triés par `OccurredOnUtc`
Tentatives avant abandon	10 (≈ 2 heures de persévérance)
Délai de base / plafond	10 s → doublement → plafonné à 30 min
Progression	10 s · 20 s · 40 s · 80 s · 160 s · 320 s · 640 s · 30 min…
Gigue	±20 % — évite le « troupeau qui charge » à la reprise d'un service

LES DEUX ERREURS SYMÉTRIQUES QUE CE MÉCANISME ÉVITE

Rejouer sans fin. Un message empoisonné — adresse e-mail invalide, type renommé, JSON illisible — occupe définitivement une place dans un lot plafonné à cinquante. À cinquante messages morts, l'outbox du module se fige : plus une commande confirmée, plus un vendeur crédité, plus une notification. Une panne qui commence par un e-mail refusé et finit par une plateforme muette.

Abandonner trop vite. Une indisponibilité de trois minutes enterrerait des messages parfaitement sains — et un message enterré, c'est un vendeur jamais payé. Le backoff exponentiel tranche : on insiste beaucoup au début, on espace ensuite, on n'abandonne qu'après deux heures.

La mise en lettre morte est journalisée en `Critical` et exposée sur `GET /admin/outbox/dead-letters`, avec rejeu manuel. C'est une perte métier réelle : elle doit être vue par un humain.

CONTRAINTE DE DÉPLOIEMENT — INSTANCE UNIQUE

La lecture du lot n'est protégée par aucun `SELECT … FOR UPDATE SKIP LOCKED`. Deux processeurs concurrents liraient les mêmes messages et les dispatcheraient deux fois : chaque gain vendeur crédité deux fois, chaque notification envoyée deux fois. D'où deux règles tenues dans le compose de production :

- les quatre BFF posent `OUTBOX_ENABLED=false` — seule l'API draine ;
- **l'API ne peut pas être mise à l'échelle horizontalement** tant que le verrou de ligne n'est pas implémenté. C'est une contrainte, pas une préférence.

Le registre `OutboxContextRegistration` est en revanche peuplé sur *tous* les hôtes, y compris ceux qui ne drainent pas : c'est ce qui permet à la console d'administration de lire et rejouer les lettres mortes des vingt-cinq modules sans jamais les traiter.

7 · Cartographie des modules

> Vingt-cinq modules composés, chacun avec son schéma PostgreSQL, ses agrégats et sa place dans la chaîne de valeur.

Module	Schéma	Fichiers	Lignes	Agrégats	Rôle
Identity	`identity`	154	10 454	User · Role · Address · PaymentMethod	Comptes, RBAC, JWT et jetons de rafraîchissement, TOTP, carnet d'adresses
Settlement	`settlement`	65	9 031	SellerWallet · PlatformWallet · SellerEarning · WalletTransaction · Withdrawal · SettlementBatch · CustomerRefund	Portefeuilles, séquestre, gains vendeurs, retraits, remboursements
Catalog	`catalog`	133	7 915	Product · Category · Brand	Catalogue de référence, médias produit, détourage d'image
Payments	`payments`	56	5 222	Payment	Encaissement, séquestre, webhooks PSP, versements
Ordering	`ordering`	42	4 137	Order	Orchestrateur du saga de commande
Sellers	`sellers`	72	3 847	Seller	Boutiques, KYB, comptes de versement, statistiques
Notifications	`notifications`	56	3 649	Notification	E-mail (Resend), push (FCM), préférences, jetons d'appareil
Inventory	`inventory`	39	3 645	InventoryItem · FulfillmentLocation	Stock, réservations, points d'expédition
Messaging	`messaging`	35	3 554	Conversation	Messagerie acheteur ↔ vendeur, temps réel SignalR
Offers	`offers`	48	2 674	Offer	Offres vendeur : prix, disponibilité, rattachement produit
Pricing	`pricing`	38	2 639	Promotion	Promotions, coupons, décomposition du prix
Shipping	`shipping`	38	2 593	Shipment · Carrier · ShippingZone	Expéditions, transporteurs, zones et forfaits
Returns	`returns`	25	2 061	ReturnRequest	Retours et remboursements
Cart	`cart`	31	1 953	Cart	Panier valorisé, clôturé à la commande
Billing	`billing`	24	1 802	Invoice · CommissionRule	Facturation et règles de commission
Reviews	`reviews`	29	1 545	Review	Avis produit et vendeur, modération
Marketing	`marketing`	18	1 496	Campaign · ContentBlock	Campagnes et contenu éditorial des vitrines

Module	Schéma	Fichiers	Lignes	Agrégats	Rôle
Disputes	`disputes`	23	1 491	Dispute	Litiges acheteur / vendeur
Search	`search`	21	1 382	read model	Index de recherche alimenté par projection d'événements
Loyalty	`loyalty`	16	1 054	LoyaltyAccount	Points de fidélité
Wishlist	`wishlist`	16	1 013	Wishlist	Listes d'envies
Tax	`tax`	18	1 001	TaxRule	Règles de taxation
Analytics	`analytics`	17	942	read model	Agrégats de mesure, instantanés
Fraud	`fraud`	17	804	RiskAssessment	Scoring de risque
Recommendations	`recommendations`	14	766	read model	Recommandations produit

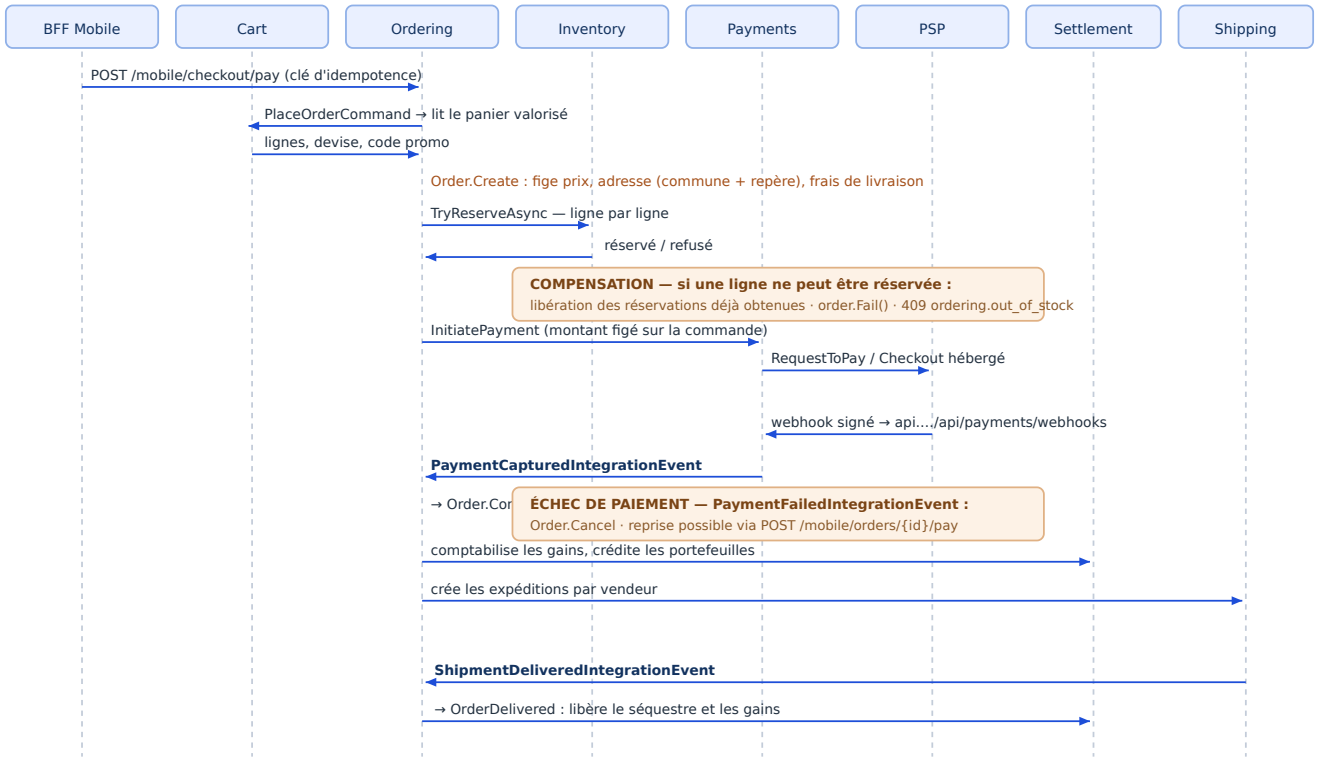
L'ARBORESCENCE DIT DÉSORMAIS LA VÉRITÉ

Jusqu'au 8 septembre 2026, `src/Modules/` comptait trente et un dossiers pour vingt-cinq modules réels. Six ne contenaient plus que des répertoires `bin/` et `obj/` — **Delivery** (`HBA.Deliveries.*`), **Food**, **Media**, **Product**, **User** et **Wallet** — vestiges d'une nomenclature antérieure dont les responsabilités avaient été absorbées : le portefeuille par `Settlement`, les médias et le traitement d'image par `Catalog`, les utilisateurs par `Identity`. Six dossiers de tests jumeaux, tout aussi vides, les accompagnaient.

Ils ont été supprimés. `src/Modules/` contient maintenant exactement vingt-cinq dossiers, un par module composé. **Delivery** et **Food** restent des périmètres à venir — un cahier des charges `CDC_Service_Livraison_Coursier.md` vit dans `docs/` — mais ils ne sont plus annoncés par une coquille vide.

8 · Le saga de commande

> Le cas d'usage le plus transverse : il traverse huit modules, comporte une étape compensable, et se poursuit longtemps après la réponse HTTP.



Les traits pleins bleus sont des appels synchrones ; les libellés en gras désignent des événements d'intégration passés par l'outbox.

Séquence complète du checkout, compensations comprises.

8.1 · Ce que l'orchestrateur garantit

- **Les prix sont figés** dans la commande à sa création. Le panier est clôturé juste après ; plus rien ne peut bouger entre deux tentatives de paiement.
- **Le code promo est reporté sur la commande.** Le coupon n'est décompté qu'à la *confirmation* (paiement encaissé) : sans ce report, Pricing recevrait un « commande confirmée » sans savoir quel coupon consommer, et le coupon resterait éternellement disponible.
- **L'adresse de livraison est un invariant du domaine**, pas une validation d'interface. Ce contrôle vivait auparavant dans un `if` du BFF Mobile — et deux autres chemins d'écriture (l'API et le BFF Web) appelaient la même commande sans jamais fournir d'adresse. Un invariant vrai sur un seul des trois chemins n'est pas un invariant.
- **Ce qui est exigé de l'adresse : la commune et le point de repère.** Pas la rue — au Bénin elle est souvent inexistante, et l'exiger reviendrait à faire inventer une adresse. Le repère est ce que le coursier utilise réellement.
- **La reprise de paiement est un chemin distinct.** `/checkout/pay` crée une commande depuis le panier ; quand le paiement échoue, la commande existe et le panier est vide, donc cette route n'est plus utilisable. `POST /mobile/orders/{id}/pay` rejoue le paiement d'une commande existante, sans clé d'idempotence puisque rien ne peut plus être dupliqué.

9 · Le flux financier

> Le module `Settlement` tient les comptes : sept agrégats, et l'obligation que la somme des parts soit exactement égale au brut encaissé.

9.1 · Le modèle de prix

Le brut d'une ligne — ce que paie l'acheteur — est construit à partir du prix net voulu par le vendeur :

```
brut = net_vendeur × (1 + taux_commission + taux_provider)
```

La répartition inverse préserve l'exactitude de la somme :

Part	Calcul	Destination
Commission plateforme	`brut × commission / (1 + total)`	solde commission de la plateforme
Frais prestataire	`brut × provider / (1 + total)`	solde provider de la plateforme
Net vendeur	`brut − commission − provider`	solde « à venir » du vendeur
Frais de livraison	figés sur la commande	solde livraison de la plateforme

Le net vendeur est calculé par soustraction, jamais par multiplication : c'est ce qui garantit que la somme des trois parts est *exactement* le brut, sans écart d'arrondi.

Les deux taux sont **figés au moment de la vente** et voyagent avec la marchandise : `Offer` les mémorise, `CartItem` les recopie, `OrderLine` les grave. Le calcul vit dans un seul endroit, `GrossSplit`, qui porte aussi son inverse — la contre-passation au prorata.

> **DEUX VERROUS DE COHÉRENCE**
>
> **Idempotence explicite.** L'outbox garantit « au moins une fois » : chaque handler financier vérifie qu'il n'a pas déjà agi avant d'écrire.
>
> **Concurrence optimiste.** Un conflit sur un portefeuille devient un **409 « réessayez »**, et non un 500 : l'un est réessayable par le client, l'autre déclenche une astreinte.
>
> **Barème figé.** Le prix était construit avec le barème du jour de la vente et réparti avec celui du jour de la confirmation : passer la commission à 12 % faisait toucher 9 830 à un vendeur qui avait fixé son net à 10 000. L'accrual applique désormais le taux de la ligne.

9.2 · Le cycle de vie de l'argent

Étape	Déclencheur	Effet
Encaissement	webhook PSP → `PaymentCaptured`	Commande confirmée, fonds sous séquestre côté `Payments`.
Comptabilisation	`OrderConfirmed`	`AccrueEarningsOnOrderConfirmedHandler` crée un `SellerEarning` par ligne et crédite le solde « à venir » du vendeur plus les trois soldes plateforme. **Idempotent par commande.**
Libération	`ShipmentDelivered` puis `OrderDelivered`	Le gain passe de « à venir » à « disponible ». En parallèle, `Payments` libère le séquestre.
Retour	`ReturnRefunded`	`ReverseEarningsOnReturnRefundedHandler` contrepasse le gain au prorata ; le remboursement client est enregistré.
Retrait	demande vendeur	`Withdrawal` → payout → `PayoutPaid` → notification. Un job de réconciliation rattrape les retraits en cours.

10 · Persistance

> Une base PostgreSQL 16, vingt-cinq schémas, cent-cinq migrations. La séparation est logique — mais elle est réelle, et testée.

10.1 · Un schéma par module

Chaque `ModuleDbContext` déclare son schéma (`HasDefaultSchema`) et y crée sa propre table `OutboxMessages`. Il en résulte vingt-cinq périmètres étanches : `analytics`, `billing`, `cart`, `catalog`, `disputes`, `fraud`, `identity`, `inventory`, `loyalty`, `marketing`, `messaging`, `notifications`, `offers`, `ordering`, `payments`, `pricing`, `recommendations`, `returns`, `reviews`, `search`, `sellers`, `settlement`, `shipping`, `tax`, `wishlist`.

Aucune clé étrangère ne traverse un schéma. Une commande référence un `SellerId` et un `ProductId` comme des identifiants opaques, pas comme des relations. Le prix de ce choix est connu : la cohérence référentielle inter-modules n'est pas garantie par la base, elle l'est par le domaine et par les événements. Le dépôt en tire d'ailleurs les conséquences — un script `scripts/orphelins-fk.sql` et une note `docs/22-severance-et-orphelins.md` traitent explicitement le sujet des orphelins.

10.2 · Migrations

Aspect	Mise en œuvre
Nombre	105 migrations EF Core, réparties par module
Application	`RunMigrationsAtStartup` — **vrai uniquement sur l'hôte `api`** ; les quatre BFF démarrent sans jamais toucher au schéma
Ordre	Séquentiel et explicite dans `Program.cs`, module par module
Amorçage	Rôles RBAC par défaut, puis premier administrateur — **idempotent** : un redéploiement ne réinitialise jamais un mot de passe existant
Vérification	Après l'amorçage, le code *relit* la base pour vérifier que l'administrateur est actif et porte bien son rôle : `AlreadyPresent` ne dit rien de l'état du compte

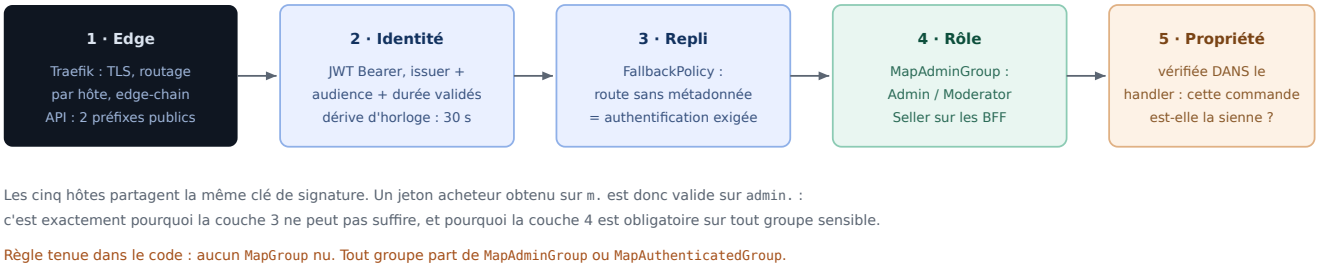
> **LE PROBLÈME « POULE / ŒUF » DE L'ADMINISTRATEUR**
>
> Sur une base neuve, créer un administrateur exige déjà d'être administrateur. Le hook d'amorçage casse la boucle : piloté par `ADMIN_EMAIL` / `ADMIN_PASSWORD`, idempotent, et suivi d'une vérification effective de connexion. Sans ces variables, aucun compte n'est créé — ce qui est légitime sur une base peuplée et bloquant sur une base neuve, d'où l'avertissement explicite au démarrage. Aucun compte de démonstration n'est jamais semé.

10.3 · Cache

`ICacheService` s'appuie sur `IDistributedCache` : cache mémoire par défaut, remplacé par Redis dès que `Redis:ConnectionString` est renseigné. Le panier et les read models chauds passent par là. Détail révélateur du soin porté aux services transverses : le logger du service de cache est résolu en *optionnel* (`NullLogger` en repli), parce qu'exiger un `ILogger` ferait échouer tout conteneur monté sans `AddLogging()` — c'est le cas de plusieurs harnais de tests qui n'installent qu'un module. Un service transverse ne doit pas imposer ses dépendances de confort à ceux qui l'utilisent.

11 · Sécurité

> Quatre couches successives, dont aucune ne suffit seule. L'ordre du pipeline est lui-même porteur de garanties : déplacer un middleware suffirait à annuler la protection tout en donnant l'apparence qu'elle est appliquée.



Les cinq hôtes partagent la même clé de signature. Un jeton acheteur obtenu sur `m.` est donc valide sur `admin.` : c'est exactement pourquoi la couche 3 ne peut pas suffire, et pourquoi la couche 4 est obligatoire sur tout groupe sensible.

Règle tenue dans le code : aucun `MapGroup` nu. Tout groupe part de `MapAdminGroup` ou `MapAuthenticatedGroup`.

La chaîne d'autorisation, de l'edge jusqu'au handler.

11.1 · Mécanismes

Mécanisme	Détail
Politique de repli	Inverse la logique par défaut d'ASP.NET : une route sans métadonnée d'autorisation exige un utilisateur authentifié. Un oubli ne crée plus une faille mais un 401, qu'on remarque immédiatement. Les routes réellement publiques déclarent `AllowAnonymous`, qui prime : webhooks PSP, retour de paiement, connexion, sondes de santé.
Garde de clé JWT	`JwtKeyGuard.EnsureProductionKey` : l'application **refuse de démarrer** en production avec la clé de développement committée dans le dépôt. Le compose impose déjà un override ; c'est la seconde barrière, pour tous les autres chemins de déploiement.
Limitation de débit	`AddAuthRateLimiter` sur les endpoints d'authentification. Partitionné par **identifiant de compte** quand l'utilisateur est authentifié (mitigation du CGNAT), sinon par IP. D'où sa position dans le pipeline : **après** `UseAuthentication`, sans quoi `HttpContext.User` est encore anonyme et la clé retombe toujours sur l'IP.
En-têtes de proxy	`UseProxyForwardedHeaders` est **le tout premier middleware**. Il restaure la vraie IP client transmise par Traefik en `X-Forwarded-For`. Placé plus bas, journalisation, authentification et limiteur liraient l'IP du proxy — une seule partition pour la planète entière.
Webhooks signés	Un webhook non signé n'est accepté qu'en développement. En production, secret absent = webhook rejeté.
Téléversements	`UploadValidation` + `FileSignature` : le type réel du fichier est vérifié par sa signature binaire, pas par son extension ni son `Content-Type`.
Mots de passe et MFA	BCrypt pour le hachage, `ITotpService` (Otp.NET) pour l'authentification à deux facteurs, `ISecureTokenGenerator` pour les jetons de vérification et de réinitialisation.
Jetons de rafraîchissement	Transportés en cookie côté consoles (`RefreshTokenCookie`) plutôt qu'en `localStorage` — sujet traité explicitement dans `docs/21-jetons-hors-du-localstorage.md`.

Swagger	Activé en développement uniquement. Aucune surface de documentation d'API en production.
Métriques de sécurité	`SecurityMetricsMiddleware` compte les 401 / 403 / 429 : le brute-force et l'énumération deviennent des séries temporelles, donc des alertes.

12 · Intégrations externes

> Chaque intégration existe en deux implémentations — réelle et simulée — sélectionnées par la présence de la configuration. Sans clé, on reste en simulation ; le développement local ne dépend d'aucun compte tiers.

12.1 · Paiements

Passerelle	Flux	Note
FedaPay	page hébergée	Encaissement et *payout* (deux passerelles distinctes). PSP de référence sur le marché béninois.
MTN Mobile Money	RequestToPay	L'acheteur approuve sur son téléphone ; le PSP notifie par callback, avec interrogation de statut en repli. Devise XOF.
Moov	mobile money	Second opérateur mobile money de la zone.
Stripe	Checkout	Cartes internationales.
PayPal	approbation	Sandbox par défaut.

Chaque option porte une propriété `IsConfigured` : identifiants absents, la passerelle simulée prend le relais. Les webhooks entrants arrivent sur l'unique surface publique de l'API et sont vérifiés par signature.

12.2 · Notifications

Canal	Mise en œuvre
E-mail	**Resend** (`ResendEmailSender`), avec `DevelopmentEmailSender` en local. Le domaine doit être vérifié (SPF + DKIM).
Push	**FCM HTTP v1** (`FcmPushSender`) via un compte de service monté en lecture seule ; `NullPushSender` si le fichier est absent — aucun push, aucune erreur.
Temps réel	**SignalR** (`ChatHub`) avec backplane Redis, pour que les pushs traversent les processus `bff-mobile` ↔ `bff-seller` .
Préférences	Table dédiée par utilisateur, plus un registre de jetons d'appareil.

> **L'API REFUSE DE DÉMARRER SANS CANAL E-MAIL EN PRODUCTION**
>
> Ce n'est pas une précaution excessive. Sans e-mail, aucun compte ne peut être vérifié et aucun mot de passe oublié ne peut être récupéré : les utilisateurs sont définitivement enfermés dehors, et l'exploitant ne l'apprend que par les plaintes. C'est ce vide qui avait conduit, à un moment de l'histoire du dépôt, à renvoyer le jeton de réinitialisation dans la réponse HTTP d'un endpoint anonyme — soit une prise de contrôle de n'importe quel compte. Mieux vaut un démarrage refusé qu'une plateforme qui « tourne » dans cet état.

12.3 · Médias et traitement d'image

- **Stockage** — Cloudflare R2, pour les médias produit (`CloudflareR2MediaStorage`) comme pour les pièces KYB des vendeurs (`CloudflareR2KybStorage`), sur des configurations distinctes.
- **Détourage** — un conteneur `rembg` 2.0.77 (modèle u2net) déployé **en réseau interne strict** : aucun port publié, aucun label Traefik. Le service n'a ni authentification ni limitation de débit, et son endpoint `GET /api/remove` va chercher une URL pour le compte de l'appelant — l'exposer serait une invitation au déni de service.

- **Aplatissement** — SkiaSharp compose le résultat détouré sur fond blanc et encode en JPEG.
- **Découplage assumé** — aucun service applicatif ne `depends_on` rembg : un `service_healthy` ferait attendre toute la pile derrière une fonction de confort. Si rembg est absent, la création de produit fonctionne, la photo n'est simplement pas détourée, et les consoles cessent de l'annoncer.

12.4 · Référentiel géographique

Le Bénin est modélisé dans le noyau partagé : douze départements, soixante-dix-sept communes, avec des codes stables en minuscules sans accent (`abomey-calavi`, `seme-podji`) stockés en base et des libellés accentués servis à l'affichage. Deux BFF exposent ce référentiel, plus un troisième côté admin. Les adresses de livraison sont construites sur **commune + quartier + point de repère**, la voie restant facultative — c'est une adaptation directe à la réalité de l'adressage local, pas une simplification.

13 · Observabilité

> Un socle unique, appelé par les cinq hôtes avec pour seule différence le nom du service. Aucun collecteur n'est requis : sans endpoint OTLP configuré, l'instrumentation tourne sans exporter.

Pilier	Mise en œuvre
Logs	Serilog, JSON compact sur `stdout`, enrichi de la propriété `service`. Prêts pour Loki ou Seq sans transformation.
Traces	OpenTelemetry — instrumentation ASP.NET Core et `HttpClient`, export OTLP.
Métriques	Instrumentation runtime + quatre familles maison : disponibilité (`api_up`, uptime, redémarrages, santé des dépendances), sécurité (401 / 403 / 429), paiements, et métier.
Sondes	`/health` (liveness) et `/health/ready` (readiness, tag `ready` sur Postgres et Redis). Les healthchecks Docker interrogent `/health` toutes les 15 s.

> **POURQUOI TOUTES LES MÉTRIQUES SONT ENREGISTRÉES SUR TOUS LES HÔTES**
>
> Ce n'est pas de la générosité mais une nécessité : chaque BFF compose les mêmes modules en processus et exécute donc les mêmes handlers, qui réclament ces compteurs par injection. N'enregistrer les métriques de paiement que sur l'API ferait échouer la résolution de dépendances sur un BFF au premier appel du handler concerné.

13.1 · Pile de supervision

Un `docker-compose.monitoring.yml` séparé assemble Prometheus (avec règles d'alerte), Alertmanager, Loki et Promtail pour les logs, Tempo pour les traces, un collecteur OpenTelemetry et Grafana. Un exporteur de requêtes PostgreSQL (`postgres-queries.yaml`) complète l'ensemble. La pile est optionnelle et découplée de l'applicatif.

13.2 · Métriques d'outbox

`OutboxMetrics` instrumente le mécanisme le plus critique : messages publiés, échecs, mises en lettre morte. Combiné au log `Critical` et à l'endpoint d'administration, c'est ce qui rend visible une classe de panne autrement silencieuse.

14 · Déploiement

> Un VPS, Docker Compose, un Dockerfile unique paramétré qui produit les cinq images. Deux environnements, deux hébergeurs, deux clés SSH — délibérément séparés.

14.1 · Construction

Un seul `Dockerfile` sert les cinq services : le projet à publier et son DLL d'entrée sont passés en arguments de build depuis le compose. Le contexte est la racine du dépôt, pour disposer de `Directory.Build.props`, `Directory.Packages.props` et de tous les modules.

```
ARG PROJECT   // ex. src/Bff/Marketplace.Bff.Mobile/Marketplace.Bff.Mobile.csproj
ARG APP_DLL   // ex. Marketplace.Bff.Mobile.dll

RUN --mount=type=cache,target=/root/.nuget/packages,sharing=locked \
    dotnet restore "${PROJECT}"
```

`sharing=locked` n'est pas un détail : BuildKit construit les cinq images en parallèle, et cinq `dotnet restore` écrivant simultanément dans le même cache NuGet échouent sur des fichiers temporaires. Le verrou sérialise l'accès au cache sans désactiver la parallélisation de la compilation.

14.2 · Services de production

Service	Image	Exposition	Particularité
`traefik`	traefik:v3.1	80 / 443	ACME Let's Encrypt, socket Docker en lecture seule, configuration dynamique par fichiers
`api`	marketplace/api	`api.` — 2 préfixes	Migrations, seed admin, **seul drain d'outbox**
`bff-mobile`	marketplace/bff-mobile	`m.`	SignalR, liens profonds, fiche App Store
`bff-web`	marketplace/bff-web	`web.`	—
`bff-seller`	marketplace/bff-seller	`seller.`	—
`bff-admin`	marketplace/bff-admin	`admin.`	Lecture et rejeu des lettres mortes
`seller-console`	marketplace/seller-console	`pro.`	Console vendeur (Next.js)
`console`	marketplace/console	`console.`	Console d'administration (Next.js)
`postgres`	postgres:16-alpine	**aucun port**	`shm_size` 256 Mo, configuration montée, limite 8 Go
`redis`	redis:7-alpine	**aucun port**	AOF activé, mot de passe requis
`rembg`	danielgatis/rembg:2.0.77	**aucun port**	Version épinglée, `--threads 2`, modèle u2net en volume nommé

Chaque hôte .NET est plafonné à 1 CPU et 640 Mo, avec un healthcheck HTTP sur `/health`. Postgres et Redis ne publient aucun port : ils ne sont joignables que sur le réseau Docker interne.

14.3 · Chaîne CI/CD

Étape	Contenu
Déclencheur	`push` sur `develop_staging` → staging ; sur `develop` → production. Concurrence groupée par branche, sans annulation.
Garde-fou	`dotnet restore` → `build -c Release` → `dotnet test` sur la solution entière. Les tests d'architecture s'exécutent ici.

Images	Build multi-projets et push sur GHCR (`ghcr.io/hectorberi01`), tag `latest` ou `staging` . Préflight explicite sur la présence du PAT.
Déploiement	Ansible sur le VPS cible ; le serveur ne fait que `pull` (`use_registry`) — il ne compile jamais.
Environnements	Staging et production sur deux hébergeurs distincts, avec **une clé SSH par environnement**. Un secret unique couplait les deux : réparer l'accès à l'un cassait silencieusement l'autre, et la panne n'apparaissait qu'à la livraison suivante.

DEUX DÉTAILS DE PRODUCTION QUI ONT CHACUN LEUR HISTOIRE

rembg épinglé en 2.0.77. L'option `--no-ui` n'existe pas avant la 2.0.75, et le correctif d'une traversée de chemin non authentifiée y est arrivé en même temps. Un `pull` ramenant une image antérieure produirait soit une boucle de redémarrage, soit une régression de sécurité silencieuse.

Le volume du modèle monté sur `/root/.u2net`. L'image amont y place déjà `u2net.onnx`. Le déplacer via `U2NET_HOME` forcerait un téléchargement de 176 Mo au premier appel — à l'intérieur de la requête d'un vendeur, donc au-delà du délai imparti.

15 · Garde-fous et stratégie de test

> Trente-cinq projets de test, 279 cas. La particularité tient moins au volume qu'à la nature de l'un d'eux : les frontières architecturales sont vérifiées par la compilation, pas par la revue.

15.1 · Tests d'architecture

Quatorze suites `NetArchTest`, regroupées par famille de modules, imposent quatre familles de règles :

Règle	Ce qu'elle empêche
`Domain_ne_doit_pas_dependre_de_EFCore`	Qu'un agrégat se mette à porter des attributs de mapping ou à connaître un `DbContext`.
`Domain_ne_doit_dependre_ni_de_Application_ni_de_Infrastructure`	L'inversion du sens des dépendances.
`Application_ne_doit_pas_dependre_de_Infrastructure`	Qu'un handler contourne son port pour appeler directement un repository EF.
`Applications_ne_touchent_que_les_Contracts_des_autres_modules`	**La règle centrale.** Un module qui atteindrait le Domain d'un autre casserait toute possibilité d'extraction.
`Contracts_ne_doit_exposer_aucun_interne`	Qu'un DTO public référence par mégarde un type interne, transformant le contrat en fuite.
`Application_ne_doit_pas_dependre_des_internes_d_Identity`	Cas particulier : Identity est le module le plus sollicité, donc le plus exposé à la contamination.

15.2 · Les autres niveaux

Niveau	Périmètre
Tests unitaires 33 projets, un par module	Invariants du domaine et logique des handlers, sans base ni conteneur. C'est ce que la séparation Domain / Application rend possible : la règle métier se teste sans infrastructure.
Tests d'intégration Catalog.IntegrationTests	Mapping EF, migrations et requêtes réelles sur PostgreSQL via Testcontainers.
Tests de saga Saga.IntegrationTests	Le chemin complet du checkout, compensations comprises — la seule manière de vérifier une chorégraphie d'événements bout en bout.
Tests d'architecture 14 suites	Les frontières. Ils échouent la CI avant tout déploiement.

> **CE QUE LA POLITIQUE DE COMPILATION AJOUTE**
>
> `TreatWarningsAsErrors` et `Nullable` activés sur toute la solution : un avertissement de nullabilité ne peut pas s'accumuler silencieusement, il bloque la build. Combiné à `InvariantGlobalization`, cela élimine deux classes entières de bugs de production — déréférencement nul et comportement dépendant de la culture système.

16 · Limites connues et trajectoire

> Ce que l'architecture actuelle ne permet pas encore, et dans quel ordre s'y attaquer. Aucun de ces points n'est un défaut de conception : ce sont des conséquences assumées de choix qui restent justes au stade actuel.

Limite	Conséquence	Levée
Outbox sans verrou de ligne `bloquant pour scaler`	L'API ne peut pas être répliquée : deux instances dispatcheraient chaque événement deux fois — chaque gain vendeur crédité en double.	Implémenter `SELECT … FOR UPDATE SKIP LOCKED` sur la lecture du lot. C'est le **premier** chantier à mener avant toute montée en charge.
Dispatch in-process	Un événement n'est publié que dans le processus qui draine ; il n'existe pas de bus partagé.	Le contrat est déjà sérialisé et nommé de façon stable : substituer un consumer Kafka ne toucherait aucun handler métier.
Clé JWT unique aux 5 hôtes	Un jeton acheteur est techniquement valide sur l'hôte d'administration ; seule la couche de rôle l'arrête.	Audiences distinctes par hôte, ou clés séparées. À mettre en regard du coût : la couche de rôle couvre le risque aujourd'hui.
Base unique	Les vingt-cinq schémas partagent une instance : une saturation les affecte tous.	La séparation par schéma rend l'extraction d'un module vers sa propre base mécanique — à faire module par module, en commençant par le plus chargé.
Barème plateforme en configuration `étape suivante`	Le choix des taux vit encore dans une section de config côté Offers, alors que `Billing` porte un agrégat `CommissionRule` bien plus capable — scope, priorité, plancher et plafond, date d'effet — dont l'API `ICommissionModuleApi` n'est appelée par personne.	Brancher `ICommissionModuleApi` dans Offers, semer une règle plateforme à 10 % / 5 % pour un comportement identique au démarrage.
Remise vendeur en double	`Offer.ApplyDiscount` et `Promotion(FundedBy.Seller)` font la même chose par deux chemins, sans règle de cumul. Le premier impose en plus un job de fond soumis à la contrainte d'instance unique.	Faire de la promotion vendeur la seule autorité, réduire `Offer` au prix vendeur, et supprimer `DiscountExpiryService`.
Delivery et Food non implémentés	Deux périmètres métier prévus (livraison coursier, restauration) n'existent qu'à l'état de cahier des charges.	Le patron de module est stable : quatre projets, un installateur, une ligne dans `ModuleRegistry`.
Search et Recommendations en base	L'index de recherche est un read model PostgreSQL alimenté par projection d'événements. Cela tient au volume actuel.	La projection étant déjà découplée, remplacer le magasin par un moteur dédié ne toucherait que l'infrastructure du module.

16.1 · Ordre recommandé

1. **Verrou de ligne sur l'outbox** — c'est le seul point qui empêche aujourd'hui de répliquer l'hôte qui porte le travail de fond, et donc le seul plafond dur de la plateforme.

2. **Supervision des lettres mortes** — l'endpoint et le log existent ; il manque l'alerte Alertmanager qui réveille quelqu'un, puisque chaque lettre morte est une perte métier.
3. **Barème unique dans Billing** — les taux sont désormais figés sur la ligne de commande ; il reste à unifier l'endroit où ils sont *choisis*. L'agrégat existe déjà, il n'est pas branché.
4. **Séparation des audiences JWT** — défense en profondeur sur la couche la plus sensible.
5. **Extraction d'un premier module** — `Notifications` est le meilleur candidat : vingt-et-un abonnements entrants, aucun appel synchrone sortant, aucune donnée partagée. Le tester validerait que la promesse d'extractibilité est réelle.

> **CE QUE L'ARCHITECTURE TIENT DÉJÀ**
>
> Les frontières ne sont pas déclaratives : elles sont testées à chaque build. Le mécanisme de cohérence entre modules a une politique de réessai bornée, une issue explicite et une visibilité d'exploitation. La surface publique est réduite à ce qui doit l'être, et l'autorisation est construite en couches dont aucune ne suppose que la précédente a été bien faite. Pour une plateforme multi-vendeurs portée par une équipe réduite, c'est le bon compromis — et il laisse la porte ouverte à la découpe le jour où elle deviendra nécessaire.

Annexe A · Événements d'intégration et abonnés

> Trente-huit événements publiés, quarante-sept abonnements. Le tableau se lit comme la carte des dépendances asynchrones réelles du système.

Événement	Producteur	Abonnés et effet
Parcours de commande		
`CartCheckedOut`	Cart	—
`OrderPlaced`	Ordering	**Cart** clôture le panier · **Notifications** informe l'acheteur
`OrderConfirmed`	Ordering	**Settlement** comptabilise les gains · **Shipping** crée les expéditions · **Pricing** décompte le coupon · **Loyalty** crédite les points · **Sellers** rafraîchit les ventes · **Analytics** agrège · **Notifications** ×2 (acheteur et vendeur)
`OrderDelivered`	Ordering	**Payments** libère le séquestre · **Settlement** libère les gains
`OrderCancelled`	Ordering	**Notifications**
Paiement et règlement		
`PaymentCaptured`	Payments	**Ordering** confirme la commande
`PaymentFailed`	Payments	**Ordering** annule la commande · **Notifications**
`PaymentRefunded`	Payments	—
`PayoutPaid`	Settlement	**Notifications** informe le vendeur
Logistique		
`ShipmentShipped`	Shipping	**Notifications**
`ShipmentDelivered`	Shipping	**Ordering** marque la commande livrée si toutes les expéditions le sont · **Settlement** libère les gains du vendeur · **Notifications**
`StockReserved`	Inventory	—
`StockDepleted`	Inventory	**Notifications** alerte le vendeur
Catalogue et recherche		
`ProductCreated`	Catalog	**Search** indexe
`ProductStatusChanged`	Catalog	**Search** met à jour
`ProductDeleted`	Catalog	**Search** désindexe
`CategoryCreated` · `BrandCreated`	Catalog	—
`OfferCreated`	Offers	**Search** indexe
`OfferPriceChanged`	Offers	**Search** met à jour le prix
`OfferDeleted`	Offers	**Search** désindexe
`PromotionCreated`	Pricing	—
Vendeurs		
`SellerRegistered`	Sellers	**Notifications** alerte l'administration
`SellerActivated`	Sellers	**Notifications**
`SellerClosed`	Sellers	**Catalog** invalide les produits · **Notifications**
`SellerReactivated`	Sellers	**Notifications**
`SellerDeleted`	Sellers	**Catalog** purge les produits
Après-vente et engagement		

Événement	Producteur	Abonnés et effet
`ReturnRefundApproved`	Returns	**Notifications**
`ReturnRefunded`	Returns	**Settlement** contrepasse le gain · **Notifications**
`DisputeOpened` · `DisputeResolved`	Disputes	**Notifications**
`ReviewPublished`	Reviews	**Search** réindexe · **Sellers** recalcule la note · **Notifications**
`ReviewRejected`	Reviews	**Search** · **Sellers**
`MessageSent`	Messaging	**Notifications** pousse la notification
Comptes		
`UserRegistered` · `UserEmailConfirmed`	Identity	—
`EmailVerificationRequested`	Identity	**Notifications** envoie le lien de vérification
`PasswordResetRequested`	Identity	**Notifications** envoie le lien de réinitialisation

Lecture — **Notifications** concentre vingt-et-un des quarante-sept abonnements et n'émet aucun événement : c'est un puits pur, et le meilleur candidat à l'extraction. **Ordering** est le seul module à la fois gros producteur et gros consommateur : c'est le cœur de la chorégraphie.